

Real-Time Memory-Aware Congestion Estimation for Q-DDCA in Large-Scale Quantum Networks

Nhatdevzizi

Independent Researcher - Experimental Preprint

Abstract—The decentralized dynamic congestion avoid routing algorithm (Q-DDCA) uses locally observed query outcomes to estimate whether a neighboring quantum node can accept an entangled pair. Because this estimate is historical, it may continue to favor a node after its quantum memory has become congested. This paper introduces a real-time memory-aware acceptance estimator that blends historical acceptance with the neighbor's current free-memory ratio. The estimator is integrated into the original Q-DDCA expected-cost routing function without adding a central controller or global network state. A paired SimQN evaluation compares historical-only Q-DDCA with the proposed estimator over sending rates $w=1$ through 30 on identical random topologies and request pairs. Across the congested range $w=10$ through 30, the proposed method raises mean total entanglement distribution rate by 9.20% and reduces mean drops by 23.79%. In the high-load range $w=20$ through 30, the EDR gain reaches 12.02%. The results show that live memory occupancy is a useful local congestion signal, while also demonstrating that a fixed mixing weight does not eliminate network-wide overload.

Index Terms—Congestion estimation, decentralized routing, entanglement distribution, Q-DDCA, quantum memory, quantum networks.

I. INTRODUCTION

Large-scale quantum networks distribute entangled pairs between remote users so that applications can exchange quantum information or establish cryptographic material. The performance of this service depends not only on path length and fidelity, but also on the small and time-varying memory capacity of intermediate quantum nodes. When several requests transmit concurrently, a shortest path can become unusable even though alternative neighbors still have free memory. Congestion therefore appears as failed admissions, repeated queries, dropped entangled pairs, and a decline in the end-to-end entanglement distribution rate (EDR).

Q-DDCA addresses this problem with decentralized, hop-by-hop routing [1]. A sender can choose among feasible neighboring nodes by combining remaining distance with a locally estimated probability that the next node accepts the pair. This design avoids a global controller and adapts routing to observed congestion. In the original mechanism, however, the acceptance estimate is derived from a finite history of query outcomes. The history is intentionally lightweight, but it reacts only after new successes or failures are observed. A node whose memory fills suddenly can consequently retain a favorable score for several routing decisions.

This work asks whether a live, locally observable resource signal can make Q-DDCA respond sooner without changing its decentralized architecture. We augment the historical acceptance estimate with the current free-memory ratio of each candidate next hop. The resulting estimator uses one tunable weight, introduces no global state, and leaves the original expected-cost routing rule intact. A paired experiment then isolates the estimator's incremental effect by comparing two Q-DDCA variants on identical random topologies, traffic requests, and sending rates.

The evaluation yields three main findings. First, low-load behavior remains effectively unchanged: across $w=1$ through 5, mean EDR rises by only 0.45%. Second, across $w=10$ through 30, real-time estimation improves mean total EDR by 9.20% and reduces mean drops by 23.79%. Third, the method improves congestion avoidance rather than removing overload; EDR eventually declines as every route experiences memory pressure. These results extend the Q-

DDCA evaluation by measuring the value of fresher local state within the same decentralized routing framework.

II. BACKGROUND AND RELATED WORK

A. Q-DDCA Congestion-Avoidance Routing

Q-DDCA models entanglement distribution as asynchronous, connectionless, hop-by-hop forwarding [1]. Quantum memories are preemptive shared resources, and multiple requests compete without reserving complete end-to-end paths. A node queries candidate neighbors before forwarding and retains a short record of query outcomes. The success ratio estimates the probability that a candidate can accept a new pair. Q-DDCA inserts that probability into an expected routing cost that balances a successful short route against the cost of exhausting the retry budget.

The published evaluation compared Q-DDCA with a shortest-path algorithm based on Dijkstra's method [3]. Its results showed that both algorithms increase EDR while the sending window is small, but Q-DDCA preserves higher EDR and fewer drops after shortest-path routes become congested. In the five-request scenario, the published Q-DDCA curve peaks near $w=12$ and then falls under network-wide load [1]. That qualitative pattern defines the comparison format used here: sending rate on the horizontal axis, total EDR and dropped pairs as the principal outcomes, and $M=10$ as the retry setting.

B. Why Historical State Can Lag

Historical query outcomes are attractive because they require little local storage and naturally smooth stochastic failures. Their weakness is age. A candidate with nine successes and one failure still appears favorable even if its last free memory is consumed immediately before routing. The next failed query will correct the history, but correction occurs after a poor decision. Quantum memory occupancy is already maintained by each node to enforce admission, so it supplies a contemporaneous congestion signal without requiring a controller or topology-wide exchange.

III. REAL-TIME MEMORY-AWARE Q-DDCA

A. Local Acceptance Estimator

Let $n_s(v)$ be the number of successful queries and $n(v)$ the number of recorded queries for candidate neighbor v . Following the implementation's smoothing rule, the historical acceptance estimate is

$$q_{\text{hist}}(v) = [n_s(v) + \text{delta}] / [n(v) + \text{delta}], \text{ delta} = 0.5. \quad (1)$$

Let c_v and C_v denote the neighbor's currently occupied and total quantum memories. The live memory utilization u_v is clamped to the interval $[0,1]$.

$$u_v = \min(1, \max(0, c_v / C_v)). \quad (2)$$

The proposed estimator blends the smoothed history with the current free-memory ratio:

$$q_{\text{hat}}(v) = \alpha q_{\text{hist}}(v) + (1 - \alpha)(1 - u_v). \quad (3)$$

The parameter α lies in $[0,1]$. Setting $\alpha=1$ exactly recovers the historical-only behavior. Setting $\alpha=0$ uses only current free memory. The evaluated real-time configuration uses $\alpha=0.5$ so that neither a single occupancy snapshot nor a short history dominates the estimate.

B. Integration with the Routing Utility

The estimator replaces only the acceptance probability in Q-DDCA's local candidate evaluation. For a pair that has already made m attempts out of the maximum M , let d_v be the remaining hop metric through neighbor v and C_drop the penalty assigned to failure. The implementation evaluates

$$y_v = [1 - (1 - q_hat(v))^{(M-m)}] d_v + (1 - q_hat(v))^{(M-m)} C_drop. \quad (4)$$

Among candidates that satisfy the loop, path-length, and exploration constraints, the node selects the smallest y_v . A full neighbor has $u_v=1$ and therefore loses the free-memory contribution immediately, even when its historical record is still strong. A lightly loaded neighbor receives the opposite adjustment. All information remains local to the current node and its directly queried neighbors.

C. Operational Properties

The modification preserves Q-DDCA's decentralized and asynchronous design. It adds constant work per candidate: one utilization calculation and a weighted sum. It also preserves backwards compatibility because $\alpha=1$ is the baseline. The main tradeoff is sensitivity to transient occupancy. A low α reacts quickly but may chase short-lived fluctuations; a high α is stable but slower. The fixed $\alpha=0.5$ experiment is therefore a controlled first test rather than a claim of universal optimality.

IV. EXPERIMENTS AND EVALUATION

A. Evaluation Platform and Paired Design

The experiment uses the SimQN discrete-event quantum-network simulator [2] and the Q-DDCA prototype implementation. A random 50-node topology is generated for each seed with independent link probability 0.1. Five source-destination requests run concurrently for 10 s. Every node exposes ten memories; links generate 1000 pairs/s with 1 ms delay and buffer size one. The query interval is 50 ms, the maximum retry count is $M=10$, and the sending rate w is swept at every integer from 1 through 30.

The comparison is paired. For each w and seed, the random generator is reset before each algorithm. The experiment verifies identical topology signatures and source-destination pairs before accepting the run. Seeds 101, 202, and 303 therefore produce matched scenarios for the historical-only baseline ($\alpha=1.0$) and the real-time estimator ($\alpha=0.5$). Each plotted point is the arithmetic mean of the three replicates.

TABLE I
EXPERIMENTAL PARAMETERS

Parameter	Value
Topology	50 nodes; link probability 0.1
Traffic	5 concurrent source-destination requests
Sending rate	$w=1, \dots, 30$ windows per hop
Retry bound	$M=10$ attempts
Memory	10 memories per node
Query / run	50 ms / 10 s
Link model	1000 pairs/s; 1 ms; buffer 1
Replicates	paired seeds 101, 202, and 303
Algorithms	$\alpha=1.0$ baseline; $\alpha=0.5$ proposed

B. Metrics

Total EDR is the number of completed end-to-end entangled pairs divided by the 10 s duration, equivalently the sum of the five per-request throughputs. The drop metric is the mean number of pairs that exhaust routing attempts during a run; the figure retains the original paper's 'dropped qubits' wording for visual consistency. Population

standard deviation across the three seeds is retained in the JSON artifact, although the small sample is not sufficient for a significance claim.

V. EXPERIMENTAL RESULTS

A. Total Entanglement Distribution Rate

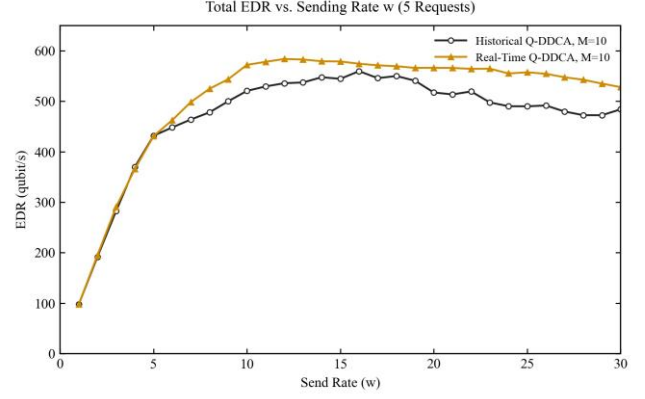


Fig. 1. Total EDR versus sending rate w for five concurrent requests. Each point is the mean of paired seeds 101, 202, and 303.

Fig. 1 shows that both variants scale similarly before congestion. Across $w=1$ through 5, the mean EDR change is only +0.45%; the proposed estimator ties the baseline at $w=1$ and is slightly lower at $w=4$ and $w=5$. Once memory pressure becomes visible, the curves separate. Across $w=10$ through 30, historical-only Q-DDCA averages 516.04 pairs/s and real-time Q-DDCA averages 563.50 pairs/s, a 9.20% increase.

The real-time curve reaches its maximum of 583.60 pairs/s at $w=12$. The historical baseline peaks later at 559.50 pairs/s at $w=16$, but cannot match the proposed peak. The largest absolute gain occurs at $w=28$, where EDR rises from 472.33 to 542.53 pairs/s, an improvement of 70.20 pairs/s or 14.86%. Across the high-load range $w=20$ through 30, the average improvement is 12.02%. The decline near the largest windows confirms that fresher routing state cannot create capacity after the network becomes broadly saturated.

B. Dropped Entangled Pairs

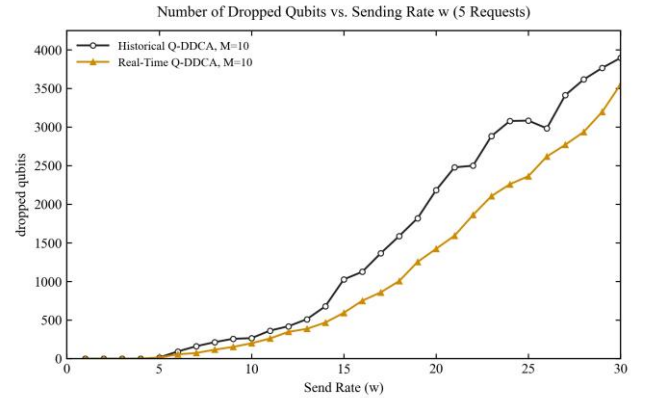


Fig. 2. Mean number of dropped entangled pairs versus sending rate w for five concurrent requests. Lower values are better.

The EDR gain is accompanied by fewer drops (Fig. 2). Over $w=10$ through 30, the mean count falls from 2049.03 to 1561.63 pairs, a 23.79% reduction. The largest absolute reduction occurs at $w=21$: historical Q-DDCA drops 2477.67 pairs and the real-time variant drops 1592.00, or 885.67 fewer pairs. At $w=30$, both variants are overloaded, but the proposed estimator still reduces drops from 3895.33 to 3553.67 while increasing EDR by 9.07%.

C. Relation to the Published Q-DDCA Evaluation

The experiment is an extension, not a reproduction of the Q-DDCA-versus-shortest-path comparison in [1]. Both curves in Figs. 1 and 2 already use Q-DDCA rerouting; the controlled factor is the freshness of the local acceptance estimate. The incremental 9.20% congested-range gain is therefore expected to be smaller than the paper's full advantage over shortest-path routing. The shape is nevertheless consistent with the original evaluation: EDR grows with w at low load, peaks around the onset of congestion, and falls when concurrency overwhelms alternative routes.

VI. DISCUSSION

A. Mechanism and Design Implications

The paired results support the intended mechanism. At low load, almost every candidate has free memory, so the live term carries little discriminating information and does not materially change EDR. At moderate and high load, occupancy varies across neighbors. The estimator can then penalize a full candidate before a new failure enters its query history, redirecting work to a path that is more likely to complete. Higher EDR and fewer drops arise together because fewer pairs spend their retry budgets on stale choices.

The result also suggests that α should be load-aware. Under low load, a high historical weight avoids reacting to harmless occupancy variation. Near congestion, a lower weight can react sooner. A practical next step is to estimate local pressure from the mean and variance of neighbor occupancy and adapt α within bounded limits. Such a controller must be evaluated for oscillation: if every sender simultaneously avoids the same busy node, traffic may shift and make another node transiently attractive.

B. Limitations

The principal limitation is statistical power. Three paired seeds provide replicate spread but not a robust confidence interval. The study also fixes $\alpha=0.5$, uses ten memories per node, and evaluates one topology family, one request count, and one link configuration. The original paper's single-request experiment used a different memory configuration, so absolute EDR values should not be compared directly. Finally, the current study omits the shortest-path baseline because its purpose is to isolate the incremental effect inside Q-DDCA rather than repeat the published baseline comparison.

A stronger evaluation should use at least tens of paired seeds, report confidence intervals and paired effect sizes, sweep α , and test memory capacities, request counts, fidelity constraints, and nonstationary traffic. Comparisons with shortest-path routing and the unmodified published settings would then separate reproduction evidence from the new estimator's effect.

VII. REPRODUCIBILITY AND ARTIFACTS

The repository provides the estimator in `entity.py`, parameter propagation in `topo.py`, the paired experiment in `exp4.py`, and focused tests in `test_congestion.py` and `test_exp4.py`. The two JSON graph artifacts embed all 60 aggregate values (30 sending rates times two algorithms), the experiment metadata, and Vega-Lite encodings. The corresponding JPEG files are the figures used in this manuscript. Because the random seeds, algorithm weights, and simulator parameters are included in the artifacts, the experiment can be rerun or extended without inferring settings from the plots.

VIII. CONCLUSION

This paper introduced a real-time memory-aware acceptance estimator for Q-DDCA. By blending a neighbor's historical query success with its current free-memory ratio, the method reacts to congestion without a controller or global state. A paired 50-node SimQN experiment shows effectively neutral behavior at low load, a 9.20% mean EDR improvement with 23.79% fewer drops over $w=10$ through 30, and a 12.02% EDR improvement over $w=20$ through 30.

The method delays the performance loss caused by stale local state but does not eliminate network-wide overload. Larger paired studies and adaptive weight selection are the clearest directions for further work.

REFERENCES

- [1] L. Chen, K. Xue, J. Li, R. Li, N. Yu, Q. Sun, and J. Lu, "Q-DDCA: Decentralized dynamic congestion avoid routing in large-scale quantum networks," *IEEE/ACM Transactions on Networking*, vol. 32, no. 1, pp. 368-381, Feb. 2024, doi: 10.1109/TNET.2023.3285093.
- [2] L. Chen et al., "SimQN: A network-layer simulator for the quantum network investigation," *IEEE Network*, early access, Jan. 23, 2023, doi: 10.1109/MNET.130.2200481.
- [3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269-271, Dec. 1959.
- [4] Nhatdevzizi, "Real-time congestion estimation extension for Q-DDCA," software and experimental artifacts, Q-DDCA repository, Aug. 2026.